MOG-ZK-PoP v1

The prime-fibre proof-of-possession system, applied to itself

Machine-checked in Lean 4 (Mathlib)

Abstract

The project builds a Schnorr-style zero-knowledge proof of possession (ZKPoP) whose *base* — the modulus that fixes its challenge arithmetic — is supplied by a mathematical object: a sporadic group order, a Golay code size, a lattice count, a number-theoretic landmark. Each object is used whole, and each of its prime fibres p^{v_p} is used again as a base of its own, so that running the fibres in parallel multiplies the challenge space up to the radical of the object.

This note records what happens when the system is turned on its own output. The corpus instantiation is itself described exactly by five integers; carried by the first five primes they form a *self-descriptor*

$$N_{\text{self}} \;=\; 2^{88}\cdot 3^{81}\cdot 5^{337}\cdot 7^{64}\cdot 11^{8},$$

an object in precisely the format the system consumes, and not one of the 88 objects it was run on. The same construction, with no special-casing, produces a proof of possession over N_{self}: complete, perfectly honest-verifier zero knowledge, knowledge-sound with error $1/2$ in a single round over the whole descriptor and $1/2310$ over its five prime fibres. Iterating — summarising *that* instance by a number and feeding it back — stops immediately: the digest of the self-instance is 27720, a fixed point of the digest map, and the digest of M_{11} and M_{12} is the fixed point 3300. Every statement below is a theorem in the accompanying Lean development and carries no unproved assumption; in particular no hardness assumption is made or used anywhere.

1 The system

Bases and the protocol

A *base* of size $n > 1$ (Lean: `RequestProject.ZKPoP.Base`) is a commutative group G together with a generator g satisfying $g^n = 1$. Writing g^x for $x \in \mathbb{Z}/n\mathbb{Z}$ via the canonical representative, the public value of a witness x is $h = g^x$ and the honest three-move transcript on randomness r and challenge c is

$$\big(a,\ c,\ s\big) \;=\; \big(g^r,\ c,\ r + cx\big), \qquad \text{accepted iff} \qquad g^s \;=\; a\,h^c.$$

The following are proved for an arbitrary base, in `RequestProject/ZKPoP.lean`.

Theorem 1.1 (Completeness)**.** *Every honest transcript is accepted.*

Theorem 1.2 (Special soundness and extraction)**.** *Two accepting transcripts (a, c, s), (a, c', s') on the same commitment whose challenge difference $c - c'$ is a unit in $\mathbb{Z}/n\mathbb{Z}$ yield, by the explicit extractor $x = (s - s')(c - c')^{-1}$, a witness for h.*

Theorem 1.3 (Perfect HVZK)**.** *The simulator's transcripts are* equal *to the real ones under the bijective reparametrisation $r \mapsto r + cx$ of the randomness, and are accepting. No statistical distance is involved.*

Theorem 1.4 (The exact challenge window)**.** *Distinct challenges below* $\mathrm{minFac}(n)$ *always have an invertible difference, and* $\mathrm{minFac}(n)$ *itself does not, so the usable single-round window has exactly* $\mathrm{cap}(n) = \mathrm{minFac}(n)$ *elements. A prover with no witness can answer at most one challenge of the window on a given commitment (`answerable_card_le_one`): the soundness error of one round is* $1/\mathrm{cap}(n)$.

Objects as bases; prime fibres as parallel bases

`RequestProject/ZKPoPBases.lean` feeds the project's corpus of 88 exactly factorised objects into this protocol, in two readings. For an object o with factorisation $\prod_i p_i^{v_i}$:

- the *whole-object* base has size $\prod_i p_i^{v_i}$ and single-round capacity cap = minFac, which is 2 for every object of even order;
- each *prime fibre* $p_i^{v_i}$ is a base whose capacity is exactly p_i, and running one protocol per fibre in parallel gives a challenge space of size $\mathrm{parcap}(o) = \prod_i p_i = \mathrm{rad}(o)$.

object	base size	cap	prime fibres	parcap
M_{11}	7 920	2	$16, 9, 5, 11$	330
M_{12}	95 040	2	$64, 27, 5, 11$	330
M_{24}	244 823 040	2	$1024, 27, 5, 7, 11, 23$	53 130
binary Golay	4 096	2	4096	2
ternary Golay	729	3	729	3
Mersenne	8 191	8 191	8191	8 191
Monster	$8.08 \cdot 10^{53}$	2	fifteen fibres	1 618 964 990 108 856 390

Table 1: Base rows, from `m11_row`, `m12_row`, `m24_row`, `binaryGolay_row`, `ternaryGolay_row`, `mersenne_row`, `monster_row`.

Theorem 1.5 (Census of bases)**.** *The* 88 *objects supply* 81 *distinct whole-object base sizes and* 337 *prime-fibre bases of* 64 *distinct sizes, and the single-round capacities take exactly* 8 *values, namely* $2, 3, 5, 7, 17, 47, 137, 8191$.

2 Applying the system to itself

The self-descriptor

The system consumes numbers-with-factorisation and returns protocols. To apply it to itself one needs its own output as such a number. Theorem 1.5 and `corpus_length` give five integers that describe the output exactly; place them as exponents over the first five primes.

Definition 2.1 (The self-descriptor)**.**

$$N_{\mathrm{self}} \;=\; 2^{88} \cdot 3^{81} \cdot 5^{337} \cdot 7^{64} \cdot 11^{8},$$

with the exponents read off as in Table 2. The corresponding object `selfObj` carries this factorisation in exactly the corpus format.

quantity produced by the system	value	prime slot
objects fed in	88	2
distinct whole-object base sizes	81	3
prime-fibre bases produced	337	5
distinct fibre base sizes	64	7
distinct single-round capacities	8	11

Table 2: The five census quantities and the primes carrying them.

Theorem 2.2 (The input really is the system's own output)**.** *`selfExps` equals the list* $\big[(2,$ *`corpus.length`*$)$, $(3, \#$*distinct base sizes*$)$, $(5, \#$*fibre bases*$)$, $(7, \#$*distinct fibre sizes*$)$, $(11, \#$*distinct capacitie computed from the corpus. The number fed back in is therefore the system's own output, not a chosen constant.*

Proposition 2.3 (The self-descriptor is an admissible input)**.** *`selfObj` is corpus-shaped — primes, strictly increasing, positive exponents — it is* not *one of the* 88 *objects the system was run on, and* $N_{\text{self}} > 1$*, so it is an admissible input.*

The base row of the system itself

Theorem 2.4 (The base row of the system itself)**.** *The system reads its own descriptor exactly as it reads* M_{11}*:*

$$\big(N_{\text{self}},\ \text{cap} = 2,\ \{2^{88}, 3^{81}, 5^{337}, 7^{64}, 11^{8}\},\ \text{parcap} = 2 \cdot 3 \cdot 5 \cdot 7 \cdot 11 = 2310\big).$$

In particular $\text{cap}(N_{\text{self}}) < \text{parcap}(N_{\text{self}})$ *(`self_parallel_gt_single`): splitting into prime fibres buys the system the same improvement it buys the sporadic groups.*

The protocol over the system's own base

Let `selfBase` be the base of size N_{self} produced by the same constructor `objBase` used for M_{11} and the Monster.

Theorem 2.5 (Completeness and perfect HVZK over the self-base)**.** *The proof of possession over `selfBase` is complete, and perfectly honest-verifier zero knowledge: real and simulated transcripts coincide under a bijection of the randomness.*

Theorem 2.6 (Extraction and soundness error over the self-base)**.** *Two accepting transcripts on the same commitment with challenges* 0 *and* 1 *yield a witness; and the window over* N_{self} *has exactly* 2 *elements, of which a prover without a witness can answer at most one. The single-round soundness error over the system's own base is* $1/2$*.*

Theorem 2.7 (The fibre-parallel protocol for the system itself)**.** *The fibre-parallel protocol over the five fibres* $2^{88}, 3^{81}, 5^{337}, 7^{64}, 11^{8}$ *is complete; a prover answering two challenge tuples on the same commitments yields a witness in every fibre whose challenges differ by a unit; the parallel challenge space has exactly* 2310 *tuples; and a prover with no witness in any fibre can answer at most one of them. The soundness error is* $1/2310$*.*

3 Iterating: the digest map and its fixed point

An instance of the protocol over an object is characterised arithmetically by three numbers: its single-round capacity, its parallel capacity, and how many parallel fibres it runs.

Definition 3.1 (The digest map)**.** For an object o with k prime fibres,

$$\text{dig}(o) \ = \ \text{cap}(o) \cdot \text{parcap}(o) \cdot (k+1),$$

the last factor counting the whole-object round alongside the k fibre rounds. A bare number is turned back into an object through its own factorisation (`factorObj`), which gives the map dig on $\mathbb{N}$ (`digestNat`): output of the system, read as input again.

Theorem 3.2 (Self-application never gets stuck)**.** *For every object with prime factorisation and size* > 1 *— every corpus object, and `selfObj` — the digest is again* > 1*, hence again a legitimate base size. The system can be applied to its own output indefinitely.*

Theorem 3.3 (The self-instance is a fixed point)**.** $\mathrm{dig}(\texttt{selfObj}) = 2 \cdot 2310 \cdot 6 = 27720$, *and* $\mathrm{dig}(27720) = 27720$. *Consequently* $\mathrm{dig}^k(27720) = 27720$ *for every* k*: applying the system to its own output any number of further times returns the same instance data.*

Theorem 3.4 (The same happens on the corpus)**.** $\mathrm{dig}(M_{11}) = \mathrm{dig}(M_{12}) = 3300$ *and* $\mathrm{dig}(3300) = 3300$. *The phenomenon of Theorem 3.3 is a property of the digest map, not a coincidence arranged for the self-descriptor.*

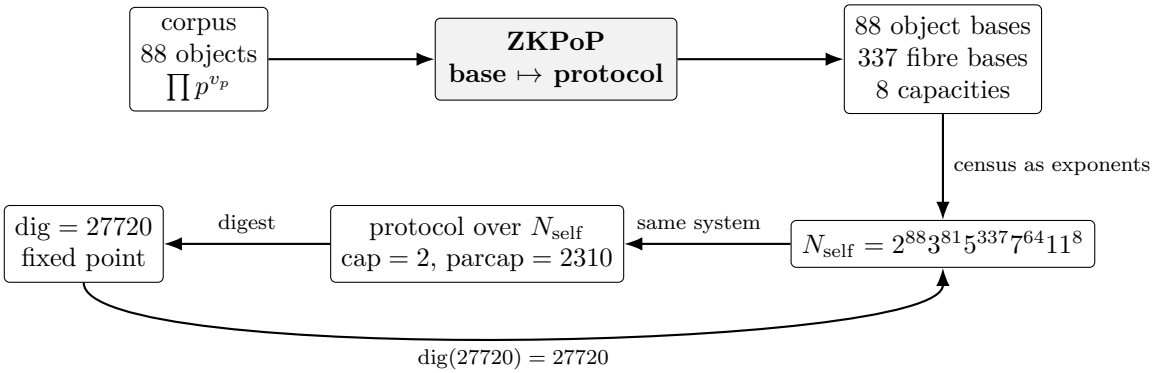

Figure 1: The loop: the corpus enters the system, the system's output is summarised by N_{self}, the same system runs on N_{self}, and the digest of the resulting instance is fixed.

4 What is and is not claimed

- **Machine-checked.** Every numbered statement above is a theorem of the Lean development, proved without `sorry`. The generic protocol is in `RequestProject/ZKPoP.lean`, the corpus instantiation in `RequestProject/ZKPoPBases.lean`, and the self-application of Sections 2–3 in `RequestProject/ZKPoPSelf.lean`. The capstone `RequestProject.ZKPoPSelf.self_application` bundles Theorems 2.2–3.3 into one statement. The only axioms used are Lean's standard ones together with the compiler-reduction axioms that the project's `native_decide` arithmetic certificates rely on.
- **No hardness assumption.** Nothing here asserts that discrete logarithms are hard in any of these bases; the standard realisation used is a group in which they are easy. What is proved is unconditional: completeness, extraction from two accepting transcripts, perfect simulatability, and the exact challenge arithmetic. These are the statements a proof-of-possession scheme is built from; instantiating them in a group where the discrete logarithm is believed hard is a separate, cryptographic, step.
- **"Applied to itself" means what it says.** The self-descriptor is derived from the system's own census by Theorem 2.2, it is not among the objects the system was run on, and the protocol over it is produced by the same functions (`baseSize`, `fibreSizes`, `singleCapacity`, `parallelCapacity`, `objBase`) that produce the protocols for M_{11}, M_{24} and the Monster. No clause of the construction was adjusted for the self case.

A Reproducing the check

From the project root, with the Lean toolchain pinned by `lean-toolchain`:

```
lake build RequestProject.ZKPoPSelf
```

and, for the axiom trace, `#print axioms RequestProject.ZKPoPSelf.self_application`.

B Lean declarations behind each statement

All names are in the `RequestProject` namespace of the accompanying project.

statement	Lean declaration(s)
Thm. 1.1 completeness	`ZKPoP.completeness`
Thm. 1.2 special soundness, extraction	`ZKPoP.special_soundness`, `ZKPoP.knowledge_soundness`
Thm. 1.3 perfect HVZK	`ZKPoP.hvzk_perfect`, `ZKPoP.sim_accepts`
Thm. 1.4 challenge window	`ZKPoP.unit_challenge_diff_of_lt_minFac`, `ZKPoP.minFac_not_unit`, `ZKPoP.window_card`, `ZKPoP.answerable_card_le_one`
Table 1 base rows	`ZKPoPBases.m11_row`, `ZKPoPBases.m12_row`, `ZKPoPBases.m24_row`, `ZKPoPBases.monster_row`
Thm. 1.5 census	`ZKPoPBases.base_census`
Def. 2.1 self-descriptor	`ZKPoPSelf.selfExps`, `ZKPoPSelf.selfObj`, `ZKPoPSelf.selfSize`
Thm. 2.2 census link	`ZKPoPSelf.selfExps_eq_census`
Prop. 2.3 admissible input	`ZKPoPSelf.selfObj_wellFormed`, `ZKPoPSelf.selfObj_not_mem_corpus`, `ZKPoPSelf.selfSize_one_lt`
Thm. 2.4 self base row	`ZKPoPSelf.self_row`, `ZKPoPSelf.self_parallel_gt_single`
Thm. 2.5 completeness, HVZK	`ZKPoPSelf.self_completeness`, `ZKPoPSelf.self_hvzk`
Thm. 2.6 extraction, error 1/2	`ZKPoPSelf.self_extraction`, `ZKPoPSelf.self_round_soundness_error`
Thm. 2.7 fibre-parallel, error 1/2310	`ZKPoPSelf.self_parallel_completeness`, `ZKPoPSelf.self_parallel_extraction`, `ZKPoPSelf.self_parallel_window_card`, `ZKPoPSelf.self_parallel_soundness_error`
Def. 3.1 digest	`ZKPoPSelf.digest`, `ZKPoPSelf.factorObj`, `ZKPoPSelf.digestNat`
Thm. 3.2 digests are inputs	`ZKPoPSelf.digest_one_lt`, `ZKPoPSelf.self_digest_one_lt`, `ZKPoPSelf.corpus_digest_one_lt`
Thm. 3.3 fixed point	`ZKPoPSelf.self_digest`, `ZKPoPSelf.digest_fixed_point`, `ZKPoPSelf.digest_orbit_const`
Thm. 3.4 corpus fixed point	`ZKPoPSelf.m11_m12_digest_fixed`
all of Sections 2–3	`ZKPoPSelf.self_application`

C The self-descriptor in full

$N_{\mathrm{self}} = 2^{88} \cdot 3^{81} \cdot 5^{337} \cdot 7^{64} \cdot 11^{8}$ has 364 decimal digits:

```
12816479704685307553423976494986368286084216648326501318987002130 9
05446408296299805323216768880884619598055279718443612750190820266 4
47630158500164711695088583408728223175657284235740711081934445818 7
71058262032584224572935816929710026081806750752889456634875386953 3
53881835937500000000000000000000000000000000000000000000000000000 0
00000000000000000000000000000000000000
```